OPERATING SYSTEMS (UE25CS242A)

Unit 2 – IPC, Threads, Synchronization and Deadlocks

Frequently Asked Questions with Answers

Lecture 1 – IPC: Shared Memory & Message Passing

Q1. Differentiate between shared memory and message passing with respect to communication mechanism and kernel involvement.

Answer:

1. In **shared memory**, communicating processes establish a region of shared memory. They exchange information by reading and writing data to the shared region. After the shared memory is established, **all accesses are treated as routine memory accesses without kernel assistance**.
2. In **message passing**, communication takes place by means of messages exchanged between processes using **send() and receive() operations**. The kernel mediates every message transfer.
3. Shared memory is **faster** for large data since there is no kernel copying after setup, but requires **explicit synchronization** by the programmer. Message passing is **easier to implement** but slower due to kernel involvement in every transfer.

Q2. Explain the Producer-Consumer problem using the bounded buffer approach.

Answer:

1. The **Producer-Consumer problem** is a paradigm for cooperating processes. A **producer** process produces information consumed by a **consumer** process.
2. In the **bounded-buffer** approach, a fixed-size buffer is shared between producer and consumer. The buffer is implemented as a **circular array** with two pointers: **in** (next free position) and **out** (first full position).
3. The buffer is **empty when in == out** and **full when ((in + 1) % BUFFER_SIZE) == out**. The producer must wait when the buffer is full; the consumer must wait when it is empty.

Q3. Differentiate between blocking and non-blocking message passing.

Answer:

1. **Blocking send**: the sender is blocked until the message is received by the receiver.
2. **Blocking receive**: the receiver is blocked until a message is available.
3. **Non-blocking send**: the sender sends the message and continues without waiting.
4. **Non-blocking receive**: the receiver receives either a valid message or a null message (if none available).
5. When both send and receive are blocking, it is called a **rendezvous** between the sender and receiver.

Q4. Compare shared memory and message passing in terms of speed, synchronization, kernel involvement, and security.

Answer:

1. **Speed**: Shared memory is faster since data is accessed directly without kernel copying. Message passing is slower due to system call overhead on every transfer.
2. **Synchronization**: Shared memory requires explicit synchronization (semaphores, mutexes) by the programmer. Message passing has built-in synchronization via send/receive.
3. **Kernel involvement**: Shared memory involves the kernel only during setup and teardown. Message passing involves the kernel for every message exchange.
4. **Security**: Message passing provides better fault isolation since processes do not share memory directly. Shared memory is vulnerable to corruption if one process writes incorrectly.

Lecture 2 – System Calls for IPC

Q5. Explain the sequence of system calls required for System V shared memory communication.

Answer:

1. **shmget()**: creates a new shared memory segment or accesses an existing one. Returns a shared memory identifier.
2. **shmat()**: attaches the shared memory segment to the virtual address space of the calling process. Returns a pointer to the shared memory region.
3. The process reads and writes to the shared memory region using the returned pointer.
4. **shmdt()**: detaches the shared memory segment from the process address space.
5. **shmctl()**: performs control operations on the shared memory segment, including marking it for destruction.

Q6. Compare ordinary pipes and FIFOs (named pipes) based on process relationship, persistence, and communication direction.

Answer:

1. **Ordinary pipes** can be used only between processes that have a **common ancestor** (typically parent-child after fork). FIFOs allow communication between **unrelated processes**.
2. Ordinary pipes exist only while the processes are running. FIFOs exist as **special files in the file system** and persist until explicitly removed using unlink().
3. Both are **half-duplex** (data flows in one direction). For bidirectional communication, two pipes/FIFOs are needed.

Q7. A programmer forgets to close the unused ends of a pipe after calling fork(). What problems can arise?

Answer:

1. If the write end of the pipe is not closed by the reader, the reader will **never receive EOF** because the OS sees an open write descriptor and assumes more data may come.
2. The reader process **blocks indefinitely** on read(), waiting for data that will never arrive.
3. If the read end is not closed by the writer, writing to a full pipe will **block the writer** since the OS cannot signal that no reader exists (SIGPIPE is not generated).

Lecture 4 – Threads, Multicore Programming & Multithreading Models

Q8. Differentiate between a process and a thread with respect to memory organization and execution.

Answer:

1. A **process** is a program in execution with its own address space — separate code, data, heap, and stack sections. Creating a process is **heavyweight** and resource-intensive.
2. A **thread** is the smallest unit of CPU execution within a process. Threads of the same process **share code, data, and open files** but have **separate stacks, registers, and program counters**.
3. Creating a thread is **lightweight** because it does not require duplicating the entire address space. Context switching between threads is faster than between processes.

Q9. Compare the Many-to-One, One-to-One, and Many-to-Many multithreading models.

Answer:

1. **Many-to-One**: multiple user threads map to a single kernel thread. Thread management is in user space (fast), but if one thread makes a blocking system call, the **entire process blocks**. No true parallelism on multicore.
2. **One-to-One**: each user thread maps to a kernel thread. Provides **true concurrency** and parallelism, but creating a thread requires creating a kernel thread, adding **overhead**. Used by Linux and Windows.
3. **Many-to-Many**: many user threads are multiplexed onto a smaller or equal number of kernel threads. Combines the benefits — **true concurrency without the overhead** of creating too many kernel threads.

Q10. Explain CPU affinity and how it conflicts with load balancing in SMP systems.

Answer:

1. **CPU affinity** binds a process or thread to a specific processor core. When a thread runs on a core, its data is loaded into that core's **cache**. Migrating the thread to another core **invalidates the cache**, causing cold-start cache misses.
2. **Soft affinity**: the OS tries to keep a thread on the same core but may migrate it. **Hard affinity**: the thread is strictly bound and never migrated.
3. **Load balancing** distributes workload evenly across cores using push migration (overloaded core pushes tasks) or pull migration (idle core pulls tasks).
4. Conflict: load balancing may **migrate a thread to a different core**, destroying cache affinity. The scheduler must balance **even workload distribution against cache performance**.

Q11. A web browser opens separate tabs for different websites. Explain why multithreading is more suitable than creating a separate process for each tab.

Answer:

1. Threads **share the code and data sections** of the process. Multiple tabs can share browser engine code, reducing **memory usage** compared to separate processes.

2. Thread creation is **faster and cheaper** than process creation because no address space duplication is needed.
3. Context switching between threads is **faster** because threads share the same address space — no need to switch page tables.
4. Threads can **communicate easily** through shared data (bookmarks, session data) without IPC mechanisms like pipes or message passing.

Lecture 7 – Mutual Exclusion and Synchronization

Q12. State the three requirements that a correct solution to the Critical Section Problem must satisfy.

Answer:

1. **Mutual Exclusion**: if a process is executing in its critical section, no other process can be executing in its critical section.
2. **Progress**: if no process is executing in its critical section and some processes wish to enter, then only those processes not executing in their remainder section can participate in deciding which enters next, and this selection **cannot be postponed indefinitely**.
3. **Bounded Waiting**: there exists a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request and before that request is granted.

Q13. Describe Peterson's Algorithm for achieving mutual exclusion between two processes.

Answer:

1. Peterson's Algorithm uses two shared variables: **flag[2]** (array indicating willingness to enter critical section) and **turn** (indicating whose turn it is).
2. To enter: Process Pi sets **flag[i] = true** and **turn = j** (gives priority to the other process). It then waits in a loop while (flag[j] == true && turn == j).
3. To exit: Process Pi sets **flag[i] = false**, allowing the other process to enter.
4. The algorithm satisfies all three requirements: **mutual exclusion, progress, and bounded waiting**. It is a purely software-based solution for two processes.

Q14. Differentiate between Test-and-Set (TSL) and Compare-and-Swap (CAS) instructions.

Answer:

1. **Test-and-Set**: atomically reads the old value of a memory location and sets it to TRUE. Returns the old value. If old value was FALSE, the caller acquires the lock.
2. **Compare-and-Swap**: atomically compares the value at a memory location with an expected value. If they match, it swaps in a new value. Returns the original value.
3. CAS is more flexible than TSL because it can be used to implement **lock-free data structures**. TSL can only set a value, while CAS can conditionally update based on the current state.

Q15. Compare software-based and hardware-supported synchronization mechanisms.

Answer:

1. Software solutions (Peterson's) require **no special hardware** but are limited to two processes and may not work on modern multicore systems due to **instruction reordering and memory model issues**.
2. Hardware solutions (TSL, CAS) are **atomic at the hardware level**, work on multicore systems, and are the foundation of all modern synchronization primitives.

3. Hardware solutions can cause **busy waiting (spinning)** which wastes CPU cycles, but are preferred for short critical sections. Software solutions avoid busy waiting but are rarely used in practice.

Lecture 8 – Mutex Locks, Semaphores, and Classical Problems

Q16. Explain the working of mutex locks and how they ensure mutual exclusion.

Answer:

1. A **mutex lock** has two states: **locked** and **unlocked**. A thread must **acquire()** the lock before entering the critical section and **release()** it when exiting.
2. If the lock is already held, the requesting thread is **blocked** until the lock becomes available. Only the thread that acquired the lock can release it (**ownership property**).
3. This ensures mutual exclusion because only one thread can hold the lock at any time, guaranteeing exclusive access to the critical section.

Q17. Develop a semaphore-based solution for the Producer-Consumer Problem.

Answer:

1. Three semaphores are used: **mutex** (initialized to 1, for mutual exclusion), **empty** (initialized to BUFFER_SIZE, counting empty slots), **full** (initialized to 0, counting full slots).
2. **Producer**: wait(empty) → wait(mutex) → add item to buffer → signal(mutex) → signal(full).
3. **Consumer**: wait(full) → wait(mutex) → remove item from buffer → signal(mutex) → signal(empty).
4. wait(empty) blocks the producer when the buffer is full. wait(full) blocks the consumer when the buffer is empty. mutex ensures only one process accesses the buffer at a time.

Q18. Explain the Dining Philosophers Problem and discuss why deadlock may occur.

Answer:

1. Five philosophers sit around a table, alternating between **thinking and eating**. Between each pair is one chopstick (five total). A philosopher needs **both adjacent chopsticks** to eat.
2. **Deadlock** occurs if all five philosophers pick up their left chopstick simultaneously. Each waits for the right chopstick, which is held by the next philosopher — a **circular wait**.
3. Solutions: allow at most four philosophers to sit, use asymmetric ordering (odd picks left first, even picks right first), or use a **monitor/mutex** to ensure a philosopher picks up both chopsticks atomically.

Q19. Compare Mutex Locks, Binary Semaphores, Counting Semaphores, and Spinlocks.

Answer:

1. **Mutex Lock**: binary lock with ownership. Only the holder can release it. Thread blocks if lock is unavailable.
2. **Binary Semaphore**: value is 0 or 1. No ownership — any thread can signal. Used for signaling between threads.
3. **Counting Semaphore**: value can be any non-negative integer. Controls access to a resource with a finite number of instances.

4. **Spinlock**: thread continuously checks (spins) the lock without blocking. Suitable for very short critical sections on multicore systems where the overhead of blocking exceeds the wait time.

Lecture 9 – Principles of Deadlock

Q20. State and explain the four necessary conditions for deadlock.

Answer:

1. **Mutual Exclusion**: at least one resource must be held in a non-sharable mode. Only one process at a time can use the resource.
2. **Hold and Wait**: a process must be holding at least one resource and waiting to acquire additional resources held by other processes.
3. **No Preemption**: resources cannot be forcibly taken away from a process. They must be released voluntarily.
4. **Circular Wait**: there exists a set of waiting processes {P0, P1, ..., Pn} such that P0 waits for a resource held by P1, P1 waits for P2, ..., Pn waits for P0.
5. All four conditions must hold simultaneously for a deadlock to occur. Preventing any one condition eliminates the possibility of deadlock.

Q21. Explain the working principle of Banker's Algorithm for deadlock avoidance.

Answer:

1. Each process must declare its **maximum resource needs** in advance. The system maintains: **Available** (resources currently free), **Allocation** (resources currently held by each process), **Maximum** (maximum demand), and **Need = Maximum - Allocation**.
2. When a process requests resources, the algorithm **pretends to grant the request** and checks if the resulting state is safe.
3. A state is **safe** if there exists a **safe sequence** — an ordering of all processes such that each process can obtain its remaining needed resources from currently available resources plus resources held by all preceding processes in the sequence.
4. If the resulting state is safe, the request is granted. If unsafe, the process must **wait** until the request can be safely satisfied.

Q22. Differentiate between deadlock prevention and deadlock avoidance.

Answer:

1. **Deadlock prevention** ensures that at least one of the four necessary conditions cannot hold. It restricts how requests can be made — for example, requiring all resources to be requested at once (preventing hold-and-wait) or imposing a total ordering on resources (preventing circular wait).
2. **Deadlock avoidance** requires advance knowledge of maximum resource needs. It dynamically examines each request and grants it only if the resulting state is **safe**. The Banker's Algorithm is the classic deadlock avoidance algorithm.
3. Prevention is **restrictive** (reduces resource utilization and throughput). Avoidance is **more flexible** but requires knowing future resource demands, which may not always be possible.

Lecture 11 – Signals and Signal Handling

Q23. Define a signal in Linux and explain how the OS handles the SIGINT signal when a user presses Ctrl+C.

Answer:

1. A **signal** is a software notification sent to a process to indicate that an event has occurred. Signals are **asynchronous** — they can arrive at any point during execution.
2. When the user presses Ctrl+C, the terminal driver generates **SIGINT** and delivers it to the foreground process.
3. The default action for SIGINT is to **terminate the process**. However, the process can register a **custom signal handler** using signal() to catch SIGINT and perform cleanup instead.

Q24. Differentiate between SIGINT, SIGTERM, SIGKILL, and SIGSTOP.

Answer:

1. **SIGINT**: generated by Ctrl+C. Requests termination. **Can be caught** and handled.
2. **SIGTERM**: default signal sent by the kill command. Requests graceful termination. **Can be caught** and handled.
3. **SIGKILL**: forces immediate termination. **Cannot be caught, blocked, or ignored**. Handled directly by the kernel.
4. **SIGSTOP**: suspends a process. **Cannot be caught, blocked, or ignored**. The process remains in memory but does not execute until SIGCONT is received.

Q25. Compare signal-based communication with pipes and shared memory.

Answer:

1. **Signals** carry **no data payload** — they are notifications only (a signal number). Pipes and shared memory can transfer **arbitrary data**.
2. Signals are **asynchronous** — they interrupt the process at any point. Pipes and shared memory require the process to actively read/write.
3. Signals require **minimal setup** (just register a handler). Pipes require file descriptors and fork(). Shared memory requires shmget(), shmat(), and synchronization.
4. Signals are suitable for **event notification and process control**. Pipes are suitable for **streaming data between related processes**. Shared memory is suitable for **high-speed bulk data exchange**.

— End of Frequently Asked Questions —